Relatório do primeiro trabalho: Isola

Grupo 9
André Vaz, nº 51585
Diogo Teixeira nº 50506
Diogo Santos nº 51846

Cadeira:
Inteligência Artificial

Abril 2026

Índice

1 Organização **3**
1.1 Diretoria `common` (Lógica Partilhada) 3
1.2 Diretoria `game` (Modos de Jogo e Controlo de Execução) 3

2 Decisões tomadas **3**
2.1 Reutilização do Turno Humano 4
2.2 Modularização do Sistema e Encapsulamento de Lógica 4

3 Principais predicados **4**
3.1 Módulo `logic.pl` 4
3.2 Módulo `io.pl` 4
3.3 Módulo `ai.pl` 5
3.4 Módulo `play.pl` 5

3.5 Módulo `start.pl` 5

4 Avaliações experimentais **5**

5 Conclusão **9**

1 Organização

O projeto foi estruturado segundo uma arquitetura modular, seguindo o princípio de *Separação de responsabilidades*, com o objetivo de garantir clareza, reutilização e facilidade de manutenção do código. A utilização da diretiva `:- module` em Prolog permitiu encapsular funcionalidades em unidades independentes, prevenindo conflitos de nomes e promovendo uma organização lógica do sistema.

A estrutura de diretórios adotada é a seguinte:

1.1 Diretoria `common` (Lógica Partilhada)

Esta diretoria agrega os componentes fundamentais do jogo, independentes do modo de execução:

- `board.pl`: Ficheiro fornecido pelo docente. Este módulo é responsável pela representação estrutural do tabuleiro. Embora não tenha sido desenvolvido pelo grupo, foi encapsulado num módulo para permitir a sua integração consistente com o restante sistema.
- `logic.pl`: Constitui o núcleo lógico do jogo. Implementa as regras formais de movimentação, remoção de peças e verificação das condições de término.
- `io.pl`: Responsável pela interação com o utilizador, incluindo leitura de coordenadas, display de mensagens e representação textual do estado do tabuleiro.

1.2 Diretoria `game` (Modos de Jogo e Controlo de Execução)

Esta diretoria contém a lógica associada à progressão do jogo e aos diferentes modos disponíveis:

- `AI/ai.pl`: Integra o algoritmo Minimax com cortes Alpha-Beta (fornecido pelo docente). Este módulo estabelece a ligação entre o modelo lógico do jogo e a árvore de pesquisa, sendo responsável pela geração de estados sucessores e decisão automática da jogada da IA.
- `human/play.pl`: Implementa o ciclo de jogo para o modo *Humano vs Humano*, incluindo validação de jogadas e alternância de turnos.
- `start/start.pl`: Ponto de entrada da aplicação. Responsável pela inicialização do jogo, apresentação do menu e encaminhamento para o modo de jogo selecionado.

2 Decisões tomadas

Durante o desenvolvimento foram tomadas algumas decisões relevantes com impacto direto na qualidade e organização do sistema:

2.1 Reutilização do Turno Humano

No desenvolvimento do modo *Humano vs Máquina*, verificou-se que a lógica associada ao turno do jogador humano era comum ao modo *Humano vs Humano*. Para evitar duplicação de código, foi criado o predicado `human_turn/3`, encapsulando toda a lógica de leitura e validação.

Desta forma, este predicado pode ser reutilizado tanto no modo tradicional como no modo com IA, garantindo um único ponto de manutenção e maior consistência no comportamento do sistema.

2.2 Modularização do Sistema e Encapsulamento de Lógica

Uma das decisões arquiteturais mais relevantes consistiu na decomposição do sistema em módulos independentes, recorrendo à diretiva `:- module/2` da linguagem Prolog. Em vez de concentrar toda a lógica num único ficheiro, optou-se por uma organização baseada no princípio de *Separation of Concerns*, promovendo uma clara divisão de responsabilidades.

A lógica central do jogo, incluindo a definição das regras e das transições válidas de estado, foi isolada no módulo `logic.pl`. Por sua vez, a representação do tabuleiro e a gestão da sua estrutura foram encapsuladas no módulo `board.pl`, enquanto a interação com o utilizador e a componente de entrada/saída foram tratadas no módulo `io.pl`.

Esta abordagem permitiu reduzir a complexidade global do sistema, evitando a poluição do *namespace* e potenciais conflitos de nomes entre predicados. Adicionalmente, o uso explícito de dependências através de `use_module/2` contribuiu para uma maior clareza estrutural, facilitando a leitura, manutenção e depuração do código.

3 Principais predicados

3.1 Módulo `logic.pl`

Este módulo define as regras fundamentais do jogo:

- `valid_move/4` e `valid_remove/3`: Verificam a validade de uma jogada. Garantem que as coordenadas estão dentro dos limites do tabuleiro e que a célula de destino contém um espaço vazio.
- `apply_move/5` e `apply_remove/4`: Produzem um novo estado do tabuleiro após a execução de uma jogada. Estes predicados respeitam o modelo imutável de Prolog, criando uma nova instância sem alterar o estado anterior.
- `has_moves/2`: Determina se um jogador possui movimentos válidos disponíveis. Este predicado é essencial para a deteção do fim do jogo.

3.2 Módulo `io.pl`

- `read_turn/4`: Responsável pela leitura estruturada do input do utilizador. Converte a entrada em coordenadas utilizáveis pelos predicados de validação.

3.3 Módulo `ai.pl`

Este módulo adapta a lógica do jogo ao formato exigido pelo algoritmo Alpha-Beta:

- `max_to_move/1` e `min_to_move/1`: Definem os papéis dos jogadores no algoritmo Minimax. A IA atua como maximizador e o humano como minimizador.
- `moves/2`: Gera todos os estados sucessores possíveis a partir de um dado estado. Utiliza `findall` para explorar todas as combinações válidas de jogadas.
- `play_ai/2`: Controla o fluxo do jogo no modo *Humano vs Máquina*. Alterna entre execução do turno humano e decisão automática da IA através do algoritmo Alpha-Beta.

3.4 Módulo `play.pl`

- `human_turn/3`: Integra leitura e validação de jogadas. Em caso de erro, repete o processo recursivamente, garantindo robustez face a inputs inválidos.
- `play/2`: Implementa o ciclo de jogo *Humano vs Humano*, incluindo alternância de turnos e verificação de término.

3.5 Módulo `start.pl`

- `start_game/0`: Inicializa o estado do jogo, apresenta o menu e direciona a execução para o modo selecionado.

4 Avaliações experimentais

Nesta secção são apresentados diferentes cenários de execução do sistema, com o objetivo de demonstrar o correto funcionamento das principais funcionalidades implementadas.

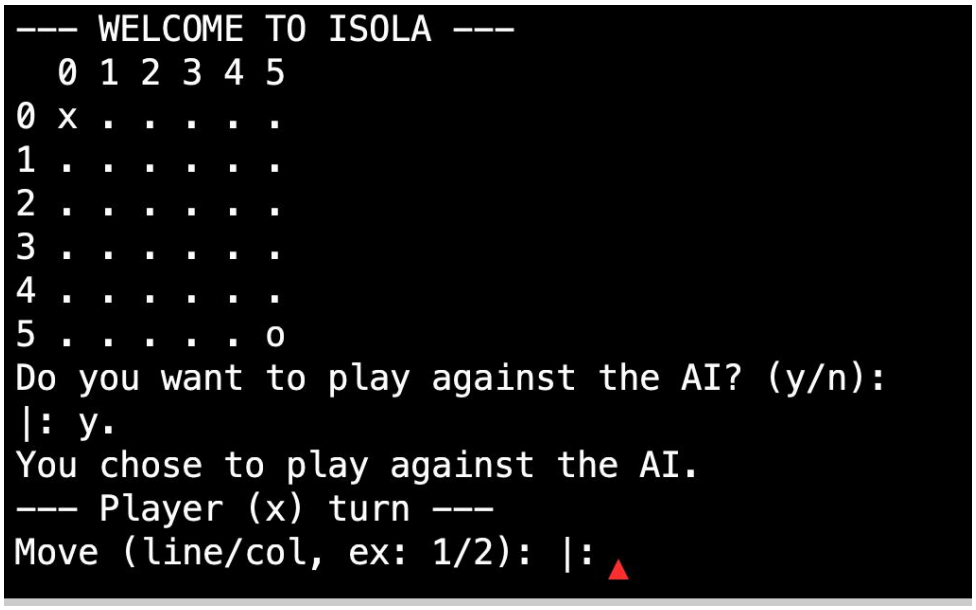

Figure 1: Seleção do modo de jogo Humano vs Máquina

```
95 ?- start_game.
--- WELCOME TO ISOLA ---
  0 1 2 3 4 5
0 x . . . . .
1 . . . . . .
2 . . . . . .
3 . . . . . .
4 . . . . . .
5 . . . . . o
Do you want to play against the AI? (y/n):
|: n.
You chose to play against another human.
Move (line/col, ex: 1/2): |:
```

Figure 2: Seleção do modo de jogo Humano vs Humano

```
Move (line/col, ex: 1/2): |: 1/1.
Remove a spot (line/col, ex: 1/2): |: 5/4.
  0 1 2 3 4 5
0 . . . . . .
1 . x . . . .
2 . . . . . .
3 . . . . . .
4 . . . . . .
5 . . . . # o
```

Figure 3: Exemplo de jogada válida (movimento e remoção)

```
  0 1 2 3 4 5
0 . . . . . .
1 . x . . . .
2 . . # . . .
3 . . . . . .
4 . . . . o .
5 . . . . # .
--- Player (x) turn ---
Move (line/col, ex: 1/2): |: 1/1.
Remove a spot (line/col, ex: 1/2): |: 3/4.
Invalid move. Try again.
Move (line/col, ex: 1/2): |:
```

Figure 4: Tentativa de movimento inválido

```
  0 1 2 3 4 5
0 . . . . . .
1 . x . . . .
2 . . # . . .
3 . . . . . .
4 . . . . o .
5 . . . . # .
--- Player (x) turn ---
Move (line/col, ex: 1/2): |: 1/1.
Remove a spot (line/col, ex: 1/2): |: 3/4.
Invalid move. Try again.
Move (line/col, ex: 1/2): |: 1/2.
Remove a spot (line/col, ex: 1/2): |: 2/2.
Invalid removal. Try again.
Move (line/col, ex: 1/2): |:
```

Figure 5: Tentativa de remoção inválida

```
Move (line/col, ex: 1/2): |: 1/1.
Remove a spot (line/col, ex: 1/2): |: 5/4.
  0 1 2 3 4 5
0 . . . . . .
1 . x . . . .
2 . . . . . .
3 . . . . . .
4 . . . . . .
5 . . . . # o
--- AI (o) turn ---
AI moved to: [4,4]
AI removed at: [2,2]
Confidence (Value): -2

  0 1 2 3 4 5
0 . . . . . .
1 . x . . . .
2 . . # . . .
3 . . . . . .
4 . . . . o .
5 . . . . # .
--- Player (x) turn ---
Move (line/col, ex: 1/2): |:
```

Figure 6: Execução do jogo contra a IA

```
  0 1 2 3 4 5
0 . # . . . .
1 x # # . . .
2 # # . . . .
3 . . . . . .
4 . . . . o .
5 . . . . . .
--- PLAYER o's TURN ---
Move (line/col, ex: 1/2): |: 4/3.
|: 0/0.
  0 1 2 3 4 5
0 # # . . . .
1 x # # . . .
2 # # . . . .
3 . . . . . .
4 . . . o . .
5 . . . . . .
PLAYER o WINS!
true .
```

Figure 7: Vitória do jogador humano

```
--- AI (o) turn ---
AI moved to: [3,5]
AI removed at: [0,0]
Confidence (Value): 100

  0 1 2 3 4 5
0 # # # . . .
1 x # . . . .
2 # # # . . .
3 . . . . . o
4 . . . . . .
5 . . . . # .
The AI has beaten you! Better luck next time...
true .
```

Figure 8: Vitória da IA

5 Conclusão

O projeto desenvolvido permitiu implementar com sucesso o jogo Isola em Prolog, aplicando os principais conceitos do paradigma lógico e garantindo uma estrutura modular clara e organizada.

Um dos pontos mais relevantes foi a integração da inteligência artificial através do algoritmo Alpha-Beta no predicado `play_ai/2`. Neste contexto, o parâmetro `MaxLevel` revelou-se essencial, uma vez que controla a profundidade da pesquisa da IA. Valores mais elevados permitem decisões mais informadas e jogadas mais fortes, mas aumentam significativamente o tempo de resposta. Por outro lado, valores mais baixos tornam o jogo mais rápido, embora com uma IA menos eficiente. Assim, foi escolhido um valor intermédio que permite um bom equilíbrio entre desempenho e qualidade das jogadas.

Relativamente às avaliações experimentais, foi possível demonstrar o correto funcionamento do sistema em vários cenários: seleção do modo de jogo, execução de jogadas válidas, deteção de jogadas inválidas e situações de vitória tanto do jogador humano como da IA. Para além disso, foi também analisado o comportamento do jogo no modo contra a IA, confirmando a correta integração do algoritmo de decisão no fluxo do jogo.

De forma geral, o sistema comporta-se como esperado, sendo robusto na validação de inputs e consistente na aplicação das regras do jogo.